


[rbnvrw](#) / [community_detection.py](#)

Last active on Jan 23

python-igraph example

[community_detection.py](#)

```

1  from igraph import *
2  import numpy as np
3
4  # Create the graph
5  vertices = [i for i in range(7)]
6  edges = [(0,2),(0,1),(0,3),(1,0),(1,2),(1,3),(2,0),(2,1),(2,3),(3,0),(3,1),(3,2),(2,4),(4,5),(4,6),(5,4),(5,6),(6,
7
8  g = Graph(vertex_attrs={"label":vertices}, edges=edges, directed=True)
9
10 visual_style = {}
11
12 # Scale vertices based on degree
13 outdegree = g.outdegree()
14 visual_style["vertex_size"] = [x/max(outdegree)*25+50 for x in outdegree]
15
16 # Set bbox and margin
17 visual_style["bbox"] = (800,800)
18 visual_style["margin"] = 100
19
20 # Define colors used for outdegree visualization
21 colours = ['#fecc5c', '#a31a1c']
22
23 # Order vertices in bins based on outdegree
24 bins = np.linspace(0, max(outdegree), len(colours))
25 digitized_degrees = np.digitize(outdegree, bins)
26
27 # Set colors according to bins
28 g.vs["color"] = [colours[x-1] for x in digitized_degrees]
29
30 # Also color the edges
31 for ind, color in enumerate(g.vs["color"]):
32     edges = g.es.select(_source=ind)
33     edges["color"] = [color]
34
35 # Don't curve the edges
36 visual_style["edge_curved"] = False
37
38 # Community detection
39 communities = g.community_edge_betweenness(directed=True)
40 clusters = communities.as_clustering()
41
42 # Set edge weights based on communities
43 weights = {v: len(c) for c in clusters for v in c}
44 g.es["weight"] = [weights[e.tuple[0]] + weights[e.tuple[1]] for e in g.es]
45
46 # Choose the layout
47 N = len(vertices)
48 visual_style["layout"] = g.layout_fruchterman_reingold(weights=g.es["weight"], maxiter=1000, area=N**3, repulserac
49
50 # Plot the graph
51 plot(g, **visual_style)

```

[simple-graph.py](#)

```

1  from igraph import *
2
3  vertices = ["one", "two", "three"]
4  edges = [(0,2),(2,1),(0,1)]
5
6  g = Graph(vertex_attrs={"label": vertices}, edges=edges, directed=True)
7
8  plot(g)

```

[styling_graph.py](#)

```

1  from igraph import *
2  import numpy as np
3

```

```

4 # Create the graph
5 vertices = ["one", "two", "three"]
6 edges = [(0,2),(2,1),(0,1)]
7
8 g = Graph(vertex_attrs={"label": vertices}, edges=edges, directed=True)
9
10 visual_style = {}
11
12 # Scale vertices based on degree
13 outdegree = g.outdegree()
14 visual_style["vertex_size"] = [x/max(outdegree)*50+110 for x in outdegree]
15
16 # Set bbox and margin
17 visual_style["bbox"] = (800,800)
18 visual_style["margin"] = 100
19
20 # Define colors used for outdegree visualization
21 colours = ['#fecc5c', '#a31a1c']
22
23 # Order vertices in bins based on outdegree
24 bins = np.linspace(0, max(outdegree), len(colours))
25 digitized_degrees = np.digitize(outdegree, bins)
26
27 # Set colors according to bins
28 g.vs["color"] = [colours[x-1] for x in digitized_degrees]
29
30 # Also color the edges
31 for ind, color in enumerate(g.vs["color"]):
32     edges = g.es.select(_source=ind)
33     edges["color"] = [color]
34
35 # Don't curve the edges
36 visual_style["edge_curved"] = False
37
38 # Plot the graph
39 plot(g, **visual_style)

```

Sign up for free (https://github.com/signup?return_to=gist)

to join this conversation on

GitHub. Already have an account? [Sign in to comment \(https://gist.github.com/login?return_to=%2Frbnvrw%2Fc2424fe3ff812da892a0\)](https://gist.github.com/login?return_to=%2Frbnvrw%2Fc2424fe3ff812da892a0)